



# Өгөгдлийг цэгцлэх tidyr

Data tidying cheat sheet — хүснэгтийн хэлбэрийг өөрчлөх, нүдийг задлах, дутуу утга, үүрлэсэн өгөгдөл

**Tidy өгөгдөл** гэдэг нь tidyverse-ийн бүх багц тууштай ажиллах, хүснэгтийн өгөгдлийг зохион байгуулах нэг арга юм. **tidyr** нь эмх замбараагүй өгөгдлийг энэ хэлбэрт оруулах, мөн эсрэгээр нь хувиргах функцуудыг агуулна.

## Tidy өгөгдөл

Хүснэгт tidy байхын гурван нөхцөл:

- Хувьсагч бүр өөрийн **баганад** байрлана.
- Ажиглалт (тохиолдол) бүр өөрийн **мөрөнд** байрлана.
- Утга бүр өөрийн **нүдэнд** байрлана.

Ингэснээр хувьсагчид вектор хэлбэрээр шууд хандах боломжтой болж, R-ийн векторжуулсан үйлдлүүд байгалийн жамаараа ажиллана.

```
table1 |> mutate(rate = cases / population)
```

Жишээнүүдэд tidyr-т багтсан table1–table5 өгөгдлийг ашиглав. Эдгээр нь нэг өгөгдлийг өөр өөр хэлбэрээр харуулдаг.

## Tibble

Tibble бол дата фреймийн орчин үеийн хувилбар (**tibble** багц). tidyverse-ийн функцууд үр дүнгээ tibble хэлбэрээр буцаадаг.

- Дэд олонлог авахад илүү чанд: x[, "a"] үргэлж tibble буцаана, x\$a бүтэн нэрээр л хандана.
- Хэвлэхдээ зөвхөн эхний 10 мөр, дэлгэцэнд багтах баганыг төрлийн хамт харуулна.

### Хэвлэх тохиргоо

**options(tibble.print\_max = n, tibble.print\_min = m, tibble.width = Inf)** Хэвлэх мөр, өргөний анхдагч утгыг өөрчилнө.

**View(x)** Бүх өгөгдлийг RStudio-д харна.

**glimpse(x)** Бүх баганыг товч харна.

### Tibble үүсгэх

**tibble(...)** Багана тус бүрээр үүсгэнэ.

```
tibble(x = 1:3, y = c("a", "b", "c"))
```

**tribble(...)** Мөр тус бүрээр үүсгэнэ.

```
tribble(
  ~x, ~y,
  1, "a",
  2, "b",
  3, "c"
)
```

**as\_tibble(x, ...)** Дата фрейм, матриц, жагсаалтыг tibble болгоно.

**enframe(x, name = "name", value = "value")** Нэртэй векторыг tibble болгоно. Эсрэг үйлдэл — **deframe()**.

**is\_tibble(x)** Tibble мөн эсэхийг шалгана.

## Хүснэгтийн хэлбэрийг өөрчлөх

Өгөгдлийн утгуудыг өөр байрлалд шилжүүлж, хүснэгтийн бүтцийг өөрчилнө.

### Урт хэлбэр рүү

**pivot\_longer(data, cols, names\_to = "name", values\_to = "value", values\_drop\_na = FALSE)** Олон баганын нэрийг нэг хувьсагчийн утга болгож, утгуудыг нь нэг баганад «хураана». Хүснэгт уртасч, нарийсна.

```
table4a
#> country   `1999` `2000`
#> Afghanistan    745    2666
#> Brazil          37737   80488
```

```
table4a |>
  pivot_longer(cols = 2:3,
               names_to = "year",
               values_to = "cases")
#> country   year cases
#> Afghanistan 1999    745
#> Afghanistan 2000   2666
#> Brazil      1999  37737
#> ...
```

### Өргөн хэлбэр рүү

**pivot\_wider(data, names\_from = name, values\_from = value)** pivot\_longer() -ийн эсрэг үйлдэл. Нэг баганын утгуудыг шинэ баганын нэр болгон «дэлгэнэ». Хүснэгт өргөсч, богиносно.

```
table2 |>
  pivot_wider(names_from = type,
              values_from = count)
#> country   year cases population
#> Afghanistan 1999    745   19987071
#> ...
```

### Нэмэлт аргуунтууд

**names\_prefix = "wk"** Нэрээс хасах (эсвэл нэмэх) угтвар.

**names\_sep = "\_"** Нэрийг хэд хэдэн хувьсагч болгон хуваах тусгаарлагч; names\_to = c("a", "b") -тэй хамт.

**names\_pattern = "(.)(.)"** Нэрийг регулярь илэрхийллээр хуваана.

**names\_to = c(".value", "year")** .value — нэрийн тэр хэсгийг утгын баганын нэр болгоно.

**values\_fill = 0** pivot\_wider() -т үүсэх дутуу утгыг бөглөнө.

**values\_fn = sum** Нэг нүдэнд олон утга тохиолдвол нэгтгэх функц.

**names\_transform = list(year = as.integer)** Үүсгэсэн баганын төрлийг хөрвүүлнэ.

## Хүснэгтийг өргөтгөх

Хувьсагчдын утгуудын бүх хослолыг гаргаж, дутуу хослолыг илрүүлнэ.

**expand(data, ...)** Зөвхөн заасан хувьсагчдын бүх боломжит хослолыг агуулсан шинэ tibble үүсгэнэ.

```
mtcars |> expand(cyl, gear, carb)
```

**complete(data, ..., fill = list())** Утгуудын дутуу хослолыг мөр болгон нэмж, бусад хувьсагчийг NA (эсвэл fill) болгоно.

```
mtcars |> complete(cyl, gear, carb)
```

**crossing(...)** Векторуудын бүх хослолын tibble (давхардлыг хасаж, эрэмбэлнэ). Мөн **expand\_grid()**.

**nesting(...)** **expand()** / **complete()** дотор зөвхөн өгөгдөлд байгаа хослолыг ашиглана.

**full\_seq(x, period)** Дутуу утгагүй бүтэн дараалал үүсгэнэ, жишээ нь **full\_seq(c(1, 5), 1)**.

## Нэг өгөгдөл — таван хэлбэр

Бүх хүснэгт Афганистан, Бразил, Хятадын 1999, 2000 оны өвчлөл (cases), хүн амыг (population) агуулна. Зөвхөн **table1** tidy.

```
table1 # tidy ✓
#> country year cases population
table2 # хувьсагчийн нэр type баганад
#> country year type count
table3 # нэг нүдэнд хоёр утга
#> country year rate ("745/19987071")
table4a # он баганын нэр болсон
#> country `1999` `2000`
table5 # он хоёр баганад хуваагдсан
#> country century year rate
```

| Асуудал                   | Шийдэл                          |
|---------------------------|---------------------------------|
| Баганын нэр нь утга       | <code>pivot_longer()</code>     |
| Мөрүүдэд хувьсагч тархсан | <code>pivot_wider()</code>      |
| Нэг нүдэнд олон утга      | <code>separate_wider_*()</code> |
| Нэг утга олон нүдэнд      | <code>unite()</code>            |

## Нарийн pivot жишээ

### Нэрнээс угтвар хасах, дутуу утгыг алгасах

```
billboard |>
  pivot_longer(
    cols = starts_with("wk"),
    names_to = "week",
    names_prefix = "wk",
    names_transform = as.integer,
    values_to = "rank",
    values_drop_na = TRUE)
```

### Нэрийг хэд хэдэн хувьсагч болгох

```
who |>
  pivot_longer(
    cols = new_sp_m014:newrel_f65,
    names_to = c("diagnosis", "gender", "age"),
    names_pattern = "new_?(.*)_?(.*)_?",
    values_to = "count")
```

`new_sp_m014` → `diagnosis = sp, gender = m, age = 014`.

### Олон утгын баганыг зэрэг дэлгэх

```
us_rent_income |>
  pivot_wider(
    names_from = variable,
    values_from = c(estimate, moe))
#> GEOID NAME estimate_income estimate_rent
#> moe_income moe_rent
```

## Урт уу, өргөн үү?

| Урт хэлбэр                                                    | Өргөн хэлбэр                                                      |
|---------------------------------------------------------------|-------------------------------------------------------------------|
| ggplot2-оор зурах, бүлгээр                                    | Хүнд уншихад, тайлангийн хүснэгтэд, матрицын тооцоонд тохиромжтой |
| хураангуйлах, загварчлахад тохиромжтой                        |                                                                   |
| Олон мөр, цөөн багана                                         | Цөөн мөр, олон багана                                             |
| <code>pivot_longer()</code>                                   | <code>pivot_wider()</code>                                        |
| Эргэлзвэл: шинжилгээнд урт хэлбэр, танилцуулгад өргөн хэлбэр. |                                                                   |



### Дутуу утгатай ажиллах

Дутуу утгыг хасах, бөглөх, солих.

```
x <- tibble(x1 = c("A", "B", "C", "D", "E"),
            x2 = c(1, NA, NA, 3, NA))
```

**drop\_na(data, ...)** Заасан баганад (эсвэл аль нэгэнд) NA агуулсан мөрүүдийг хасна.

```
x |> drop_na(x2)
#> x1 x2
#> A  1
#> D  3
```

**fill(data, ..., .direction = "down")** NA-г хамгийн сүүлийн (эсвэл дараагийн) дутуу биш утгаар бөглөнө. Чиглэл: "down", "up", "downup", "updown".

```
x |> fill(x2)
#> A 1 · B 1 · C 1 · D 3 · E 3
```

**replace\_na(data, replace)** Баганаар нь NA-г тодорхой утгаар солино.

```
x |> replace_na(list(x2 = 2))
#> A 1 · B 2 · C 2 · D 3 · E 2
```

Далд дутуу утга (огт байхгүй мөр)-ыг complete() -ээр ил болгоно. Ил NA-г drop\_na() далд болгоно.

### Нүдийг задлах, нэгтгэх

Эдгээр функцээр нэг нүдэнд олон утга агуулсан, эсвэл нэг утга хэд хэдэн нүдэнд хуваагдсан баганыг засна.

#### Нэгтгэх

**unite(data, col, ..., sep = "\_", remove = TRUE, na.rm = FALSE)** Хэд хэдэн баганыг нэг багана болгон нийлүүлнэ.

```
table5 |>
  unite(century, year, col = "year", sep = "")
#> 19 + 99 → 1999
```

### Олон багана болгох

**separate\_wider\_delim(data, cols, delim, names, ...)** Нэг баганыг тусгаарлагчаар хэд хэдэн багана болгоно.

```
table3 |>
  separate_wider_delim(rate, delim = "/",
    names = c("cases", "pop"))
#> "745/19987071" → 745 | 19987071
```

**separate\_wider\_position(data, cols, widths)** Тэмдэгтийн байрлалаар хуваана.

```
table5 |> separate_wider_position(year,
  widths = c(cent = 2, yy = 2))
```

**separate\_wider\_regex(data, cols, patterns)** Регуляр илэрхийллээр хуваана.

```
df |> separate_wider_regex(x,
  c(letter = "[A-Z]", "-", number = "\\d+"))
```

### Олон мөр болгох

**separate\_longer\_delim(data, cols, delim)** Нүдийг тусгаарлагчаар хувааж, хэсэг бүрийг тусдаа мөр болгоно.

```
table3 |> separate_longer_delim(rate, delim =
  "/")
```

**separate\_longer\_position(data, cols, width)** Тогтмол өргөнөөр хувааж мөр болгоно.

### Асуудалтай мөрүүд

**too\_few = "align\_start"** Хэсэг дутуу бол NA-гаар нөхнө. Мөн "align\_end", "debug".

**too\_many = "merge"** Хэсэг илүү бол сүүлийн баганад нийлүүлнэ. Мөн "drop", "debug".

Хуучин separate() болон extract() функцууд ажилласаар байгаа боловч separate\_wider\_\*( ) гэр бүлийг ашиглахыг зөвлөж байна.

### Үүрлэсэн өгөгдөл (nested data)

**Үүрлэсэн дата фрейм** нь нэг буюу хэд хэдэн жагсаалт-багана (list-column) агуулна. Жагсаалт-баганын нүд бүрт бүхэл бүтэн дата фрейм, загвар, вектор гэх мэт дурын объект хадгалж болно.

#### Үүрлэсэн өгөгдөл үүсгэх

**nest(data, ...)** Бүлэг тус бүрийн мөрүүдийг жагсаалт-баганад дэд хүснэгт болгон «боож» хадгална.

```
n_storms <- storms |>
  group_by(name) |>
  nest()
#>   name      data
#>   Amy    <tibble [31 × 12]>
#>   Bob    <tibble [...]>
```

Бүлэглэлгүйгээр: storms |> nest(data = c(year:long)). Мөн .by = name аргумент ашиглаж болно.

#### Жагсаалт-баганатай tibble үүсгэх

**tibble(max = c(3, 4, 5), seq = list(1:3, 1:4, 1:5))** Жагсаалт-багана бүхий tibble.

**tribble(~max, ~seq, 3, 1:3, 4, 1:4)** Мөр мөрөөр.

**mutate(df, fits = list(lm(...)))** mutate() -ээр жагсаалт-багана үүсгэнэ.

**summarise(df, q = list(quantile(x)))** Хураангуй жагсаалтаар хадгална.

### Үүрийг задлах

**unnest(data, cols, ..., keep\_empty = FALSE)** Жагсаалт-баганыг задалж, элемент бүрийг мөр ба багана болгоно.

```
n_storms |> unnest(data)
```

**unnest\_longer(data, col, values\_to = NULL, indices\_to = NULL)** Жагсаалт-баганын элемент бүрийг шинэ мөр болгоно.

```
starwars |> select(name, films) |>
  unnest_longer(films)
```

**unnest\_wider(data, col)** Жагсаалт-баганын элемент бүрийг шинэ багана болгоно.

**hoist(.data, .col, ..., .remove = TRUE)** Жагсаалт-баганаас сонгосон бүрэлдэхүүнийг л шинэ багана болгон «сугалж» авна.

```
df |> hoist(metadata,
  author = "author", year = list("date", 1))
```

JSON зэрэг олон түвшинтэй өгөгдлийг unnest\_wider() → unnest\_longer() → hoist() дарааллаар tidy болгоно.

### Үүрлэсэн өгөгдөл дээр ажиллах

Жагсаалт-баганад purrr-ийн map() гэр бүлийн функцийг mutate() дотор хэрэглэнэ.

```
n_storms |>
  mutate(
    n = map_int(data, nrow),
    fit = map(data,
      \ (d) lm(wind ~ pressure, d)),
    r2 = map_dbl(fit,
      \ (m) summary(m)$r.squared)
  )
```

Ажлын урсгал: nest() → map()-ээр жагсаалт-багана дээр тооцоолох → unnest()-ээр задлах эсвэл хураангуйг гаргаж авах.

### Бусад хэрэгтэй функц

**uncount(data, weights)** Давтамжийн баганаар мөр бүрийг олшруулна; count() -ийн эсрэг үйлдэл.

```
tibble(x = c("a", "b"), n = c(1, 2)) |>
  uncount(n)
#> a · b · b
```

**chop(data, cols)** Бүлэг бүрийн утгуудыг жагсаалт-багана болгон «цавчина». Эсрэг нь – unchop().

**pack(data, ...)** Хэд хэдэн баганыг нэг дата фрейм-багана болгоно. Эсрэг нь – unpack().

**nest\_by(df, g)** Мөр бүр нь нэг бүлэг байх rowwise үүрлэсэн хүснэгт (dplyr).

### Цэгцлэх ажлын урсгал

- **Харах:** glimpse(), View() – мөр, багана юуг илэрхийлж байгааг ойлгох.
- **Тодорхойлох:** хувьсагч, ажиглалт, утга тус бүр хаана байгааг тогтоох.
- **Хэлбэржүүлэх:** pivot\_longer() / pivot\_wider().
- **Задлах:** separate\_wider\_\*( ), unite().
- **Дутуу утгыг шийдэх:** complete(), fill(), drop\_na(), replace\_na().
- **Шалгах:** count(), distinct(), anti\_join() -оор алдаа, давхардал илрүүлэх.

Нэг ажиглалтын нэгжийг (жишээ нь, улс-он) тодорхой болговол аль нь хувьсагч болохыг шийдэхэд амар болно.

### JSON-ыг tidy болгох

```
repos <- tibble(json = gh_repos)
repos |>
  unnest_longer(json) |>
  unnest_wider(json) |>
  select(id, full_name, owner, stars =
    stargazers_count) |>
  hoist(owner, login = "login")
```

Эхлээд мөр бүрийг нэг объект болгож (unnest\_longer), дараа нь талбар бүрийг багана болгож (unnest\_wider), шаардлагатай дотоод талбарыг сугалж (hoist) авна. gh\_repos өгөгдөл repurrsive багцад бий.